Estructuras de Datos

2026
Primer Cuatrimestre

https://shorturl.at/cQ65m

Repaso

Clase anterior

- Genericidad paramétrica de clases
- TDA
  - TDA Pila
  - TDA Cola

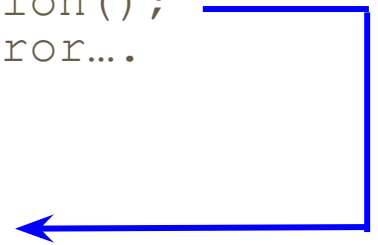
Excepciones

Excepciones: haciendo código robusto

- Las excepciones son eventos inesperados que ocurren durante la ejecución de un programa
  - Puede ser el resultado de una condición de error o una entrada incorrecta
- En POO las excepciones son objetos “lanzados” por un código que detecta una condición inesperada
- La excepción es “capturada” por otro código que repara la situación
  - Si la excepción no es capturada, entonces se propaga al método llamante y así sucesivamente hasta que algún método capture la excepción (con un try-catch)
  - Si el main no captura la excepción, el programa termina su ejecución con error

Excepciones: try-catch

```
void método( parámetros formales )
{
     try
     {
        … otras sentencias que no producen error …
        sentencia_que_puede_producir_excepción();
        … más sentencias que no producen error….
     }
     catch( tipo_excepcion e )
     {
        código para manejar la excepción e
     }
} //SI NO HAY ERROR, NO SE EJECUTA EL CÓDIGO DEL CATCH
```



Ejemplo

```java
import java.util.Scanner;
import java.io.*;

public class EjExcepcion01
{
    public static void main( String [] args )
    {
        try {
            int [] a = { 1, 2, 3, 4, 5 };
            Scanner s = new Scanner( System.in );
            System.out.print( "Ingrese una posicion entre 0 y 4: " );
            int pos = s.nextInt();
            System.out.println( "El valor en " + pos + " es " + a[pos] );
            System.out.println( "El doble es " + 2 * a[pos] );
            System.out.println( "El triple es " + 3 * a[pos] );
        } catch( ArrayIndexOutOfBoundsException e ) {
            System.out.println( "Valor fuera de rango" );
        }
    }
}
```

```
>java EjExcepcion01
Ingrese una posicion entre 0 y 4: 40
Valor fuera de rango
```

Ejemplo 2

```
try {
    int [] a = { 1, 2, 3, 4, 5 };
    Scanner s = new Scanner( System.in );
    System.out.print( "Ingrese una posicion entre 0 y 4: " );
    int pos = s.nextInt();
    System.out.println( "El valor en " + pos + " es " + a[pos] );
    System.out.println( "El doble es " + 2 * a[pos] );
    System.out.println( "El triple es " + 3 * a[pos] );
} catch( ArrayIndexOutOfBoundsException e ) {
    System.out.println( "Valor fuera de rango" );
} catch( InputMismatchException e ) {
    System.out.println( "No ingreso un entero" );
} finally {
    System.out.println( "Adios!" );
}
```

```
>java EjExcepcion02
Ingrese una posicion entre 0 y 4: mama
No ingreso un entero
Adios!

>java EjExcepcion02
Ingrese una posicion entre 0 y 4: 2
El valor en 2 es 3
El doble es 6
El triple es 9
Adios!
```

Excepciones: jerarquía de herencia

- Las excepciones se estructuran en una jerarquía de herencia
- Catch múltiples:
  - Se comprueban en forma secuencial
  - Se chequea que el tipo de la excepción conforme a la excepción declarada en el catch
    - De ser verdadero el chequeo, se ejecuta dicho bloque catch
    - De ser falso, se continúa con el catch siguiente
- ¿Cómo hay que diseñar la estructuras de catch?
  - Catch de excepciones más específicas a más generales

Ejemplo

```
try {
      …
} catch( Exception e ) {
      …
} catch( IOException e ) {
      …
}
```

IOException: indica que falló o se interrumpió una operación de entrada/salida (acceder a un archivo que no existe, se pierde conexión de red, etc)
Exception: superclase de todas las excepciones



Como IOException es subclase de Exception, **el catch de IOException es inalcanzable.**
Se debería invertir el orden de los catch

Excepciones: *checked / unchecked*

Java tiene dos tipos de excepciones:

Checked exceptions:

- Se verifican en tiempo de compilación
- El programador está obligado a capturarlas (usar un try-catch) o declarar la excepción con *throws*
- Errores esperables o recuperables (IOExceptions, SQLException)

```
import java.io.*;

public void leerArchivo(String ruta) {
    FileReader fr = new FileReader(ruta); // puede lanzar IOException
}
```



Excepciones: *checked / unchecked*

Java tiene dos tipos de excepciones:

Checked exceptions:

- Se verifican en tiempo de compilación
- El programador está obligado a capturarlas (usar un try-catch) o declarar la excepción con *throws*
- Errores esperables o recuperables (IOExceptions, SQLException)

```
import java.io.*;

public void leerArchivo(String ruta) throws IOException {
    FileReader fr = new FileReader(ruta); // puede lanzar IOException
}
```



Excepciones: *checked / unchecked*

Java tiene dos tipos de excepciones:

Unchecked exceptions:

- Ocurren en tiempo de ejecución
- El programador no está obligado a capturarlas
- Representan errores de programación (NullPointerException, ArrayIndexOutOfBoundsException

```
public void ejemplo() {
    String s = null;
    System.out.println(s.length()); // NullPointerException
}
```



Para los TDAs, trabajaremos mayormente con unchecked exceptions

Cómo saber si la excepción es *checked* o *unchecked*

Depende de la jerarquía de clases

- Si extiende *RunTimeException* → **unchecked**
- De lo contrario (solo extiende *Exception* pero no *RunTimeException*) → **checked**
- En la práctica hay que mirar el código o la documentación del método
- El IDE/compilador también nos avisa si no capturamos una excepción *checked*

Como crear nuestras excepciones

- Los métodos deben especificar en su signatura/encabezado las excepciones que pueden lanzar/propagar:

  tipo método(parámetros formales) throws clase_excepción

- Una excepción se lanza desde una sentencia throw:

  throw new clase_excepción(parámetros actuales);

- Las excepciones no manejadas en método, se propagan.
- **Ejemplo**: Programar una clase persona. Una persona tiene un nombre y una edad. La edad debe ser un número positivo.
  - Crearemos una excepción PersonaException para lanzar cuando el cliente ingresa una edad negativa en el constructor.

```java
// PersonaException.java
public class PersonaException extends Exception {
    public PersonaException( String msg ) {
        super( msg );
    }
}
// Persona.java
public class Persona {
    private String nombre;
    private int edad;

    public Persona( String nombre, int edad ) throws PersonaException {
        this.nombre = nombre;
        if( edad <= 0 )
            throw new PersonaException( "Edad negativa: " + edad );
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
    public String toString() { return getNombre() + " " + getEdad(); }
}
```

```java
// EjException03.java
public class EjException03
{
    public static void main( String [] args )
    {
        try {
            Persona p = new Persona( "Juan", 30 );
            System.out.println( "p: " + p );
            Persona q = new Persona( "Tito", p.getEdad() - 35 );
            System.out.println( "q: " + q );
        } catch( PersonaException e ) {
            System.out.println( e.getMessage() );
        }
    }
}
```

Propagación de excepciones

Ambos tipos de excepciones propagan si no se capturan (try-catch) subiendo por la pila de llamados

- hasta que alguien las captura
- O el programa termina (con error)

Ejemplo:

main → methodA → methodB → methodC

Supongamos que methodC dispara una excepción

Propagación no chequeada

```
public class UncheckedPropagation {
   public static void main(String[] args) {
      methodA();
   }
   static void methodA() {
      methodB();
   }
   static void methodB() {
      methodC();
   }
   static void methodC() {
      int x = 10 / 0; // ArithmeticException
   }
}
```

Propagación chequeada

```
public class CheckedPropagation {
   public static void main(String[] args) throws IOException {
      methodA();
   }
   static void methodA() throws IOException {
      methodB();
   }
   static void methodB() throws IOException {
      methodC();
   }
   static void methodC() throws IOException {
      throw new IOException("Something went wrong");
   }
}
```

TDA Pila / ADT Stack

NUESTRO TDA Pila

```
1  package TDAPila;
2
3  /**
4   * Interface Stack
5   * @author Catedra de Estructuras de Datos, Departamento de Cs. e Ing. de la Computaci□n,
6   */
7
8  public interface Stack<E> {
9
10     /**
11      * Consulta la cantidad de elementos de la pila.
12      * @return Cantidad de elementos de la pila.
13      */
14     public int size();
15
16     /**
17      * Consulta si la pila est□ vac□a.
18      * @return Verdadero si la pila est□ vac□a, falso en caso contrario.
19      */
20     public boolean isEmpty();
21
22     /**
23      * Examina el elemento que se encuentra en el tope de la pila.
24      * @return Elemento que se encuentra en el tope de la pila.
25      * @throws EmptyStackException si la pila est□ vac□a
```

NUESTRO TDA Pila

```
18      * @return Verdadero si la pila est vac a, falso en caso contrario.
19      */
20     public boolean isEmpty();
21
22     /**
23      * Examina el elemento que se encuentra en el tope de la pila.
24      * @return Elemento que se encuentra en el tope de la pila.
25      * @throws EmptyStackException si la pila est vac a.
26      */
27     public E top();
28
29     /**
30      * Inserta un elemento en el tope de la pila.
31      * @param element Elemento a insertar.
32      */
33     public void push(E element);
34
35     /**
36      * Remueve el elemento que se encuentra en el tope de la pila.
37      * @return Elemento removido.
38      * @throws EmptyStackException si la pila est vac a.
39      */
40     public E pop();
41 }
```

TDA Pila / ADT Stack

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3

java.util

Class Stack<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractList<E>
      java.util.Vector<E>
        java.util.Stack<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess

---

```
public class Stack<E>
extends Vector<E>
```

The Stack class represents a last-in-first-out (LIFO) stack of objects. It extends class Vector with five operations that allow a vector to be treated as a stack. The usual push and pop operatic stack, a method to test for whether the stack is empty, and a method to search the stack for an item and discover how far it is from the top.

When a stack is first created, it contains no items.

A more complete and consistent set of LIFO stack operations is provided by the Deque interface and its implementations, which should be used in preference to this class. For example:

```
Deque<Integer> stack = new ArrayDeque<Integer>();
```

Since:

JDK1.0

See Also:

Serialized Form

Field Summary

TDA Pila / ADT Stack

Method Summary

All Methods	Instance Methods	Concrete Methods

Modifier and Type	Method and Description
boolean	**empty**() Tests if this stack is empty.
E	**peek**() Looks at the object at the top of this stack without removing it from the stack.
E	**pop**() Removes the object at the top of this stack and returns that object as the value of this function.
E	**push**(E item) Pushes an item onto the top of this stack.
int	**search**(Object o) Returns the 1-based position where an object is on this stack.

Ejemplo uso de java.util.Stack

```
import java.util.Stack;
public class StackEjemplo {
        public static void main(String[] args) {
    // Creamos una pila de enteros
    Stack<Integer> stack = new Stack<Integer>();
    // Ponemos elementos en la pila
    stack.push(10);
    stack.push(20);
    stack.push(30);
    System.out.println("Imprimimos la pila: " + stack);
    // Sacamos elementos de la pila
    while (!stack.isEmpty()) {
       int value = stack.pop();
       System.out.println("Elemento sacado: " + value);
    }
    // la próxima linea arroja EmptyStackException
    // stack.pop();
  }
}
```

Implementaciones de pilas

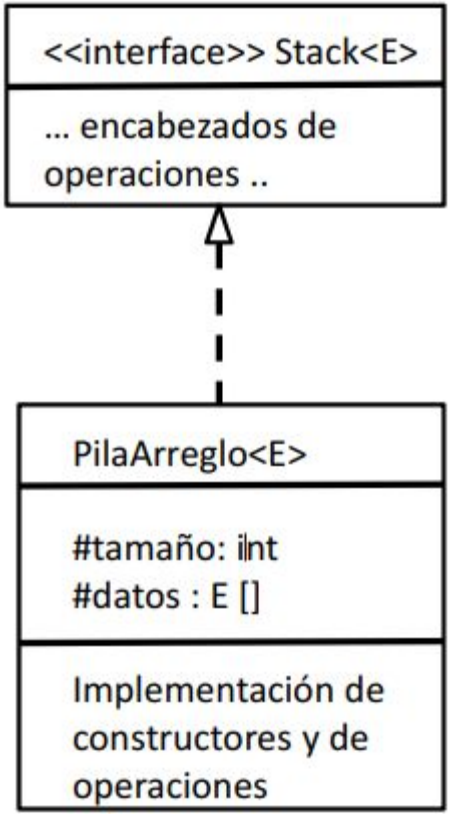
1. Usando un arreglo
2. Con una estructura de nodos enlazados
3. En términos de una lista (lo dejamos pendiente hasta ver el TDA Lista)

Implementemos el TDA Pila (ED) con Arreglos



Implementación de pila con arreglos

Usaremos una clase PilaArreglo<E> y una clase cliente UsaPilaArreglo



Excepciones para la pila

- Usaremos subclases de RuntimeException.
- Esto permite escribir código usando try-catch para capturar la excepción o propagarla si no la usamos.

```
public class EmptyStackException extends RuntimeException {

    public EmptyStackException(String mensajeDeError) {
        super(mensajeDeError);
    }
}
```

- Recuerde que heredamos los métodos:
  - `public String getMessage(); // Recupera el mensaje de error`
  - `public void printStackTrace(); // Imprime pila de registros de activación con el número de línea que falló en cada método`

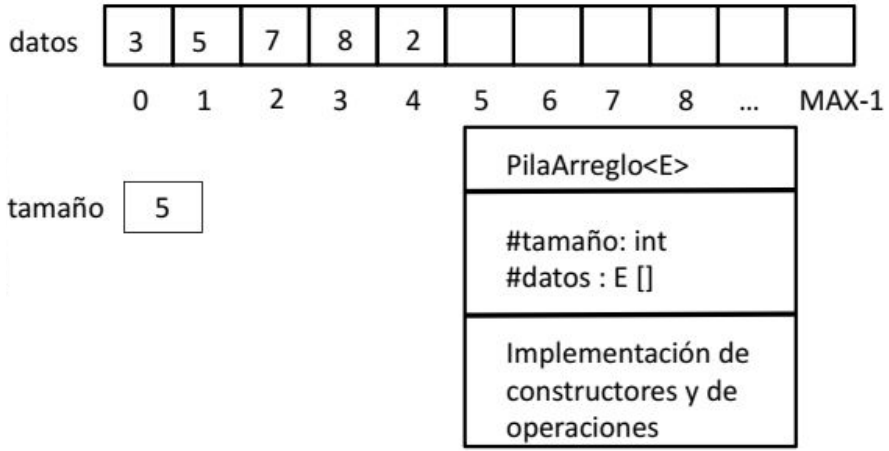



```java
//Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
        public static void main( String [] args ) {
                Stack<Integer> s = new PilaArreglo<Integer>();
                s.push( 3 );
                s.push( 5);
                s.push( 7 );
                s.push( 8 );
                s.push( 2 );
        }
}
```

Representación interna

2
8
7
5
3

```
//Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
        public static void main( String [] args ) {
                Stack<Integer> s = new PilaArreglo<Integer>();
                s.push( 3 );
                s.push( 5);
                s.push( 7 );
                s.push( 8 );
                s.push( 2 );
        }
}
```



Constructor crear una pila vacia

Constructor 1: Recibe el tamaño máximo (es responsabilidad del programador elegirlo)

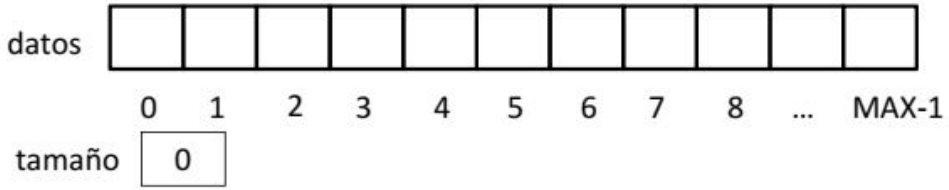
```
PilaArreglo( MAX : int ) {
        tamaño = 0;
        datos = (E []) new Object[MAX];
}
```

Constructor 2: Usa un tamaño por defecto.

```
PilaArreglo() {
        this(20);
}
```

El constructor2 crea una pila que tiene 20 elementos y delega en el constructor1.

Nota: En la implementación Java, si MAX es negativo se puede optar lanzar una excepción



Implementación: isEmpty()

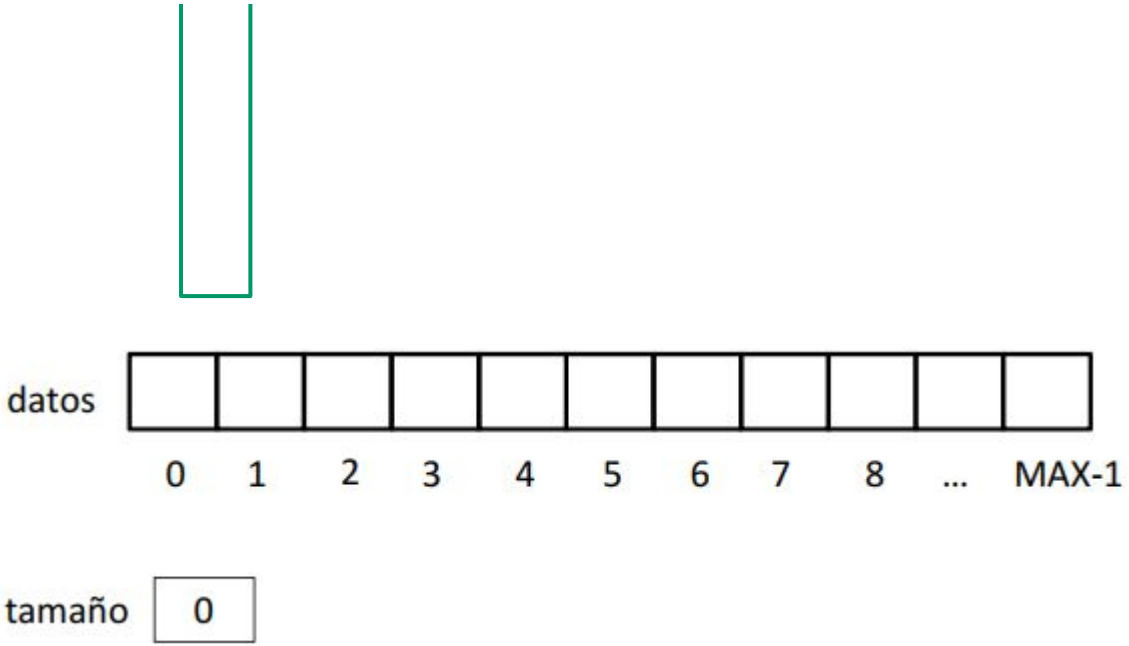
```
Pila Vacía:

isEmpty() : boolean

{

    Retornar tamaño== 0

}
```



Implementación: push(E item)

Opción 1: Si hay lugar, insertar el nuevo elemento al final del arreglo. Si no haylugar, generar una situación de error lanzando una excepción FullStackException.

Pseudocódigo:

```
Apilar(item : E)
    Si tamaño == TAMAÑO DEL ARREGLO entonces
        error (“Pila llena”)
    Sino
        datos[tamaño] = item
        tamaño = tamaño + 1
```

Java:

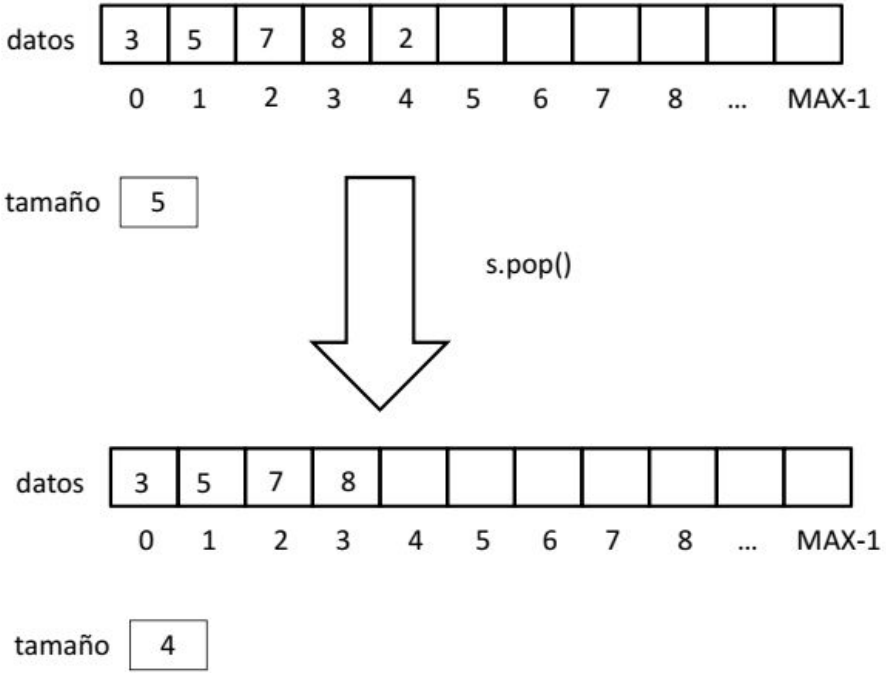
```
public void push(E item) {
    if (tamaño == datos.length)
        throw new
        FullStackException(“Pila
        llena”);
    else datos[tamaño++] = item;
}
```

Sobre apilar con pila llena

- El cliente de la pila debe determinar el tamaño con la cual crearla para evitar que se llene
- Cuando la pila se llena, tenemos 2 estrategias:
  - 1. Lanzar una excepción indicando que la estructura está llena (FullStackException)
  - 2. Aumentar el tamaño de la colección subyacente de datos
    - crear un nuevo arreglo
    - copiar los datos del arreglo original de datos en el nuevo,
    - hacer que el arreglo nuevo sea el arreglo de datos, el recolector de basura se lleva el arreglo viejo
- 1 es más fácil de programar
  - pero genera problemas al cliente al decidir el tamaño inicial y cuando la estructura se llena
- 2 es más difícil de programar
  - pero le da más flexibilidad al cliente para crear la estructura y la estructura tiene un tamaño no acotado
- 1 no tiene sobrecarga en ejecución porque siempre tarda lo mismo
- 2 tiene sobrecarga en ejecución cuando se llena porque debe generar un nuevo arreglo y copiar los datos en él
  - También requiere de un parámetro adicional
    PilaConArreglo(maximoTamaño : int; incrementoTamaño : int);

Implementación: pop() (desapilar)

2
8
7
5
3



8
7
5
3

2←s.pop()

Implementación pop (y top)

```
top() : E
     Si isEmpty() entonces
          error (“Pila vacía”)
     Sino
          retornar datos[tamaño-1]
```

```
pop() : E
     resultado top()
     datos[tamaño-1] = null
     tamaño = tamaño – 1
     retornar resultado
```

Notas: El error se modela con excepción EmptyStackException.
Si falla top(), entonces falla pop()

TDA Cola / ADT Queue

NUESTRO TDA Cola

```
1  package TDACola;
2  /**
3   * Interface Queue
4   * @author Catedra de Estructuras de Datos, Departamento de Cs. e Ing. de la Computaci□n,
5   */
6
7  public interface Queue <E>
8  {
9      /**
10      * Devuelve la cantidad de elementos en la cola.
11      * @return Cantidad de elementos en la cola.
12      */
13     public int size();
14
15     /**
16      * Consulta si la cola est□ vac□a.
17      * @return Verdadero si la cola est□ vac□a, falso en caso contrario.
18      */
19     public boolean isEmpty();
20
21     /**
22      * Inspecciona el elemento que se encuentra en el frente de la cola.
23      * @return Elemento que se encuentra en el frente de la cola.
24      * @throws EmptyQueueException si la cola est□ vac□a.
25      */
```

NUESTRO TDA Cola

```
17      * @return Verdadero si la cola est vac a, falso en caso contrario.
18      */
19     public boolean isEmpty();
20
21     /**
22      * Inspecciona el elemento que se encuentra en el frente de la cola.
23      * @return Elemento que se encuentra en el frente de la cola.
24      * @throws EmptyQueueException si la cola est vac a.
25      */
26     public E front();
27
28     /**
29      * Inserta un elemento en el fondo de la cola.
30      * @param element Nuevo elemento a insertar.
31      */
32     public void enqueue(E element);
33
34     /**
35      * Remueve el elemento en el frente de la cola.
36      * @return Elemento removido.
37      * @throws EmptyQueueException si la cola est vac a.
38      */
39     public E dequeue();
40 }
```

TDA Cola / ADT Queue

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.util

Interface Queue<E>

Type Parameters:
`E` - the type of elements held in this collection

All Superinterfaces:
`Collection<E>`, `Iterable<E>`

All Known Subinterfaces:
`BlockingDeque<E>`, `BlockingQueue<E>`, `Deque<E>`, `TransferQueue<E>`

All Known Implementing Classes:
`AbstractQueue`, `ArrayBlockingQueue`, `ArrayDeque`, `ConcurrentLinkedDeque`, `ConcurrentLinkedQueue`, `DelayQueue`, `LinkedBlockingDeque`, `LinkedBlockingQueue`, `SynchronousQueue`

```
public interface Queue<E>
extends Collection<E>
```

A collection designed for holding elements prior to processing. Besides basic `Collection` operations, queues provide additional insertion, extraction, and inspection opera operation fails, the other returns a special value (either `null` or `false`, depending on the operation). The latter form of the insert operation is designed specifically for use operations cannot fail.

Summary of Queue methods

	Throws exception	*Returns special value*
Insert	`add(e)`	`offer(e)`
Remove	`remove()`	`poll()`
Examine	`element()`	`peek()`

Queues typically, but do not necessarily, order elements in a FIFO (first-in-first-out) manner. Among the exceptions are priority queues, which order elements according to which order the elements LIFO (last-in-first-out). Whatever the ordering used, the *head* of the queue is that element which would be removed by a call to `remove()` or `poll` of queues may use different placement rules. Every `Queue` implementation must specify its ordering properties.

The `offer` method inserts an element if possible, otherwise returning `false`. This differs from the `Collection.add` method, which can fail to add an element only by throw normal, rather than exceptional occurrence, for example, in fixed-capacity (or "bounded") queues.

The `remove()` and `poll()` methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy methods differ only in their behavior when the queue is empty: the `remove()` method throws an exception, while the `poll()` method returns `null`.

TDA Cola / ADT Queue

Method Summary

All Methods | **Instance Methods** | **Abstract Methods**

Modifier and Type	Method and Description
boolean	**add**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
E	**element**() Retrieves, but does not remove, the head of this queue.
boolean	**offer**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
E	**peek**() Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll**() Retrieves and removes the head of this queue, or returns `null` if this queue is empty.
E	**remove**() Retrieves and removes the head of this queue.

Implementada en java.util.LinkedList

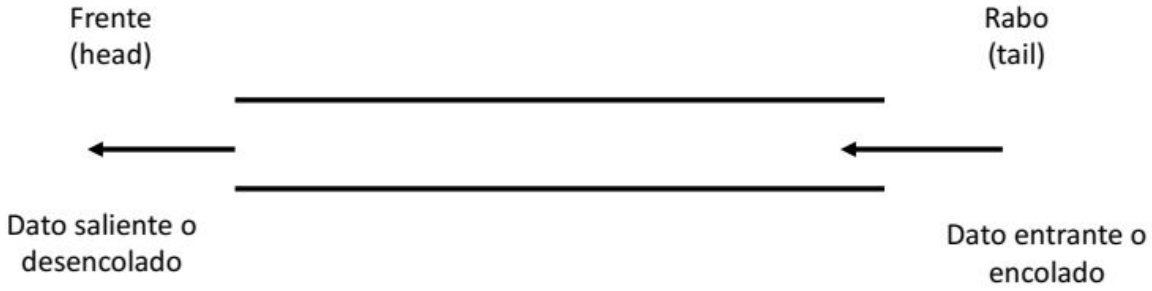
Ejemplo de uso java.util.Queue / java.util.LinkedList

```
public class QueueEjemplo {
  public static void main(String[] args) {
      Queue<Integer> queue = new LinkedList<>();
      // Agregar elementos
      queue.add(10);
      queue.add(20);
      queue.add(30);
      System.out.println("Cola inicial: " + queue);
      // Usando remove() (lanza excepción si está vacía)
      System.out.println("\nUsando remove():");
      try {
          while (true) {
              int value = queue.remove();
              System.out.println("Removed: " + value);
          }
      } catch (Exception e) {
          System.out.println("Excepción al usar remove(): " + e);
      }
      // Volvemos a llenar la cola
      queue.add(100);
      queue.add(200);
      // Usando poll() (NO lanza excepción)
      System.out.println("\nUsando poll():");
      Integer value;
      while ((value = queue.poll()) != null) {
          System.out.println("Polled: " + value);
      }
      System.out.println("Queue vacía, poll() devolvió null sin excepción.");
  }
}
```

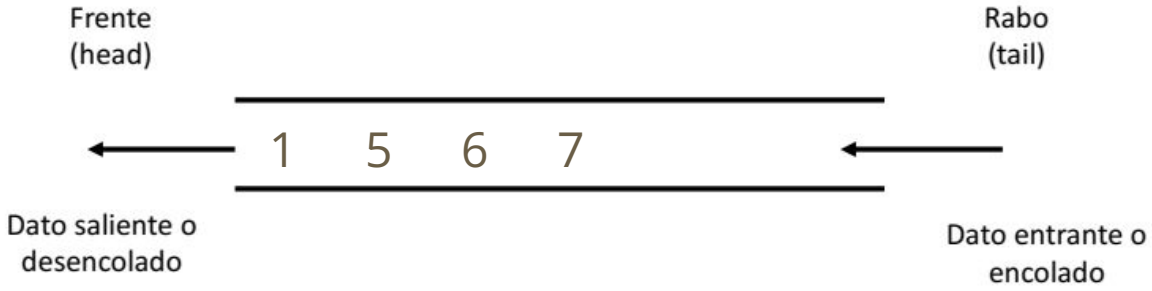
Implementemos el TDA Cola (ED) con Arreglos



TDA Cola


Frente
(head)
Rabo
(tail)
Dato saliente o
desencolado
Dato entrante o
encolado

TDA Cola




```
Queue<Integer> q = new ColaArregloCircular <Integer>();

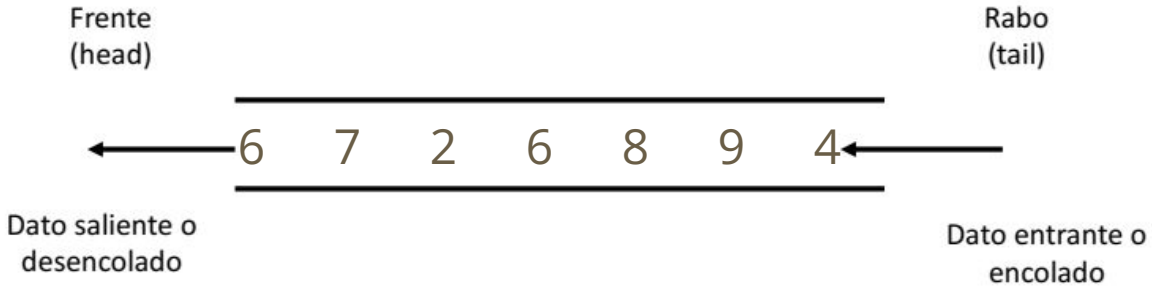
q.enqueue(1);

q.enqueue(5);

q.enqueue(6);

q.enqueue(7);
```

TDA Cola




```
Queue<Integer> q = new ColaArregloCircular <Integer>();

q.enqueue(1);

q.enqueue(5);

q.enqueue(6);

q.enqueue(7);
```

```
…

q.enqueue(8);

int elemento = q.dequeue();//elemento tiene 1
                           y se borra de la cola

q.dequeue(); // borro el 2 de la cola y no lo almaceno

q.enqueue(4); // Puedo encolar cuando quiera
```

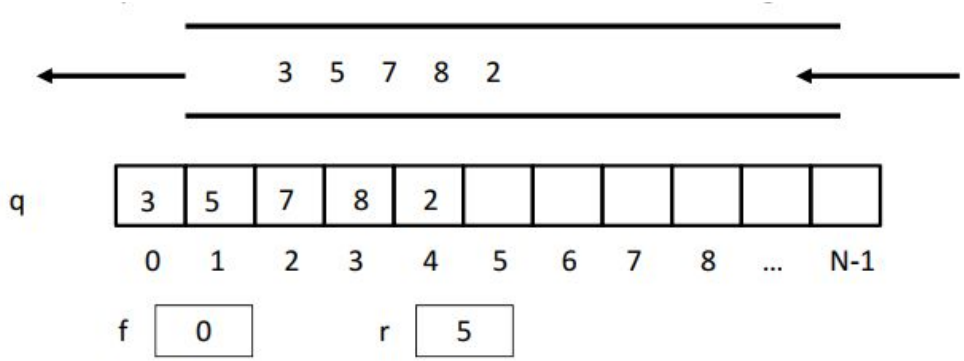
Implementaciones de cola

1) Con un arreglo circular

2) Con una estructura enlazada (más adelante)

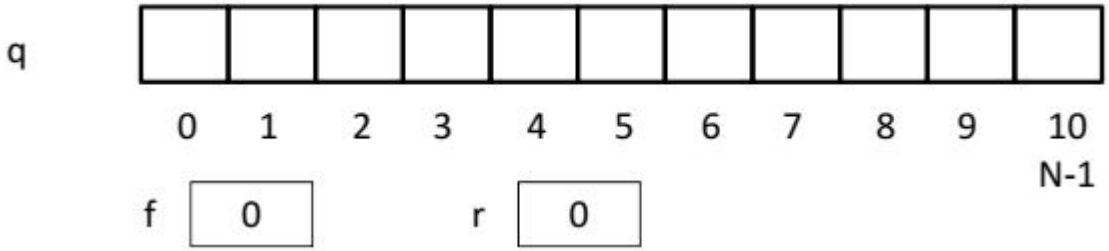
3) En términos de una lista (lo dejamos pendiente hasta ver el TDA Lista)

Implementación de cola con un arreglo circular



- q es un arreglo de N componentes y mantiene los elementos de la cola
- El tamaño máximo de q es N-1 (lo que permite diferenciar la cola vacía de la cola llena)
- f es la posición en q del próximo elemento a eliminar en un dequeue
- r es la posición en la cual se va a insertar el siguiente elemento con un enqueue

Cola con arreglo circular: Constructor



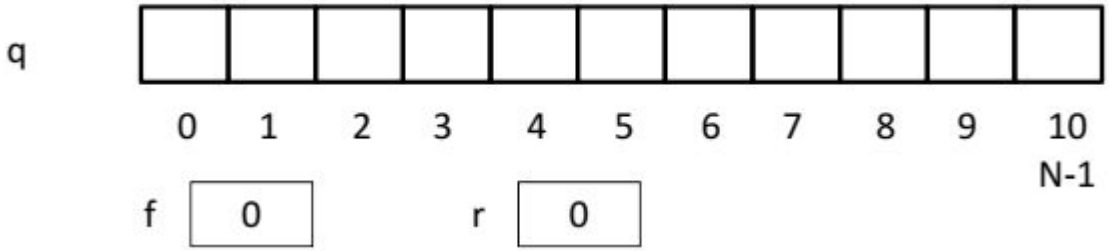
Constructor

q ← CreacionDeArreglo (N)

f = 0

r =0

Cola con arreglo circular: isEmpty()



Constructor:

q ← CreacionDeArreglo (N)

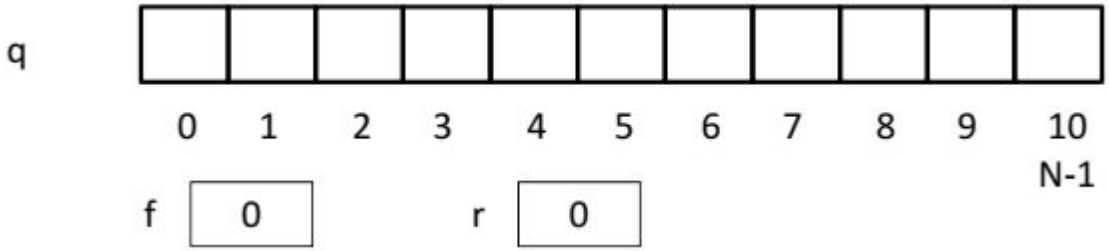
f = 0

r =0

isEmpty()

retornar (f==r)

Cola con arreglo circular: front()



Constructor:

q ← CreacionDeArreglo (N)

f = 0

r =0

isEmpty():

retornar (f==r)

front():

Si isEmpty() entonces

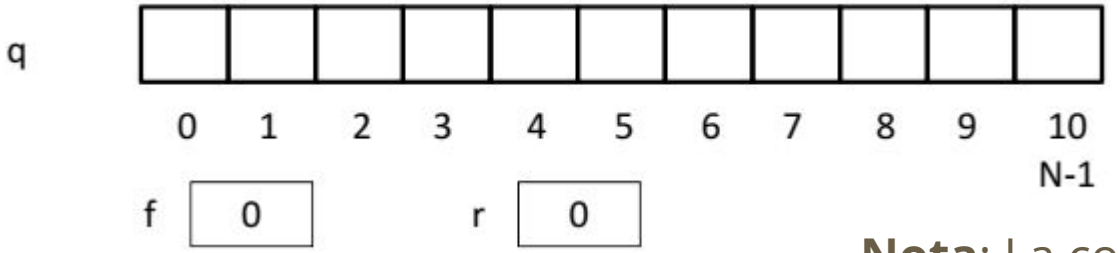
error (“Cola vacía”)

sino

retornar (q[f])

Al implementar front se debería lanzar una excepción

Cola con arreglo circular: size()



size():

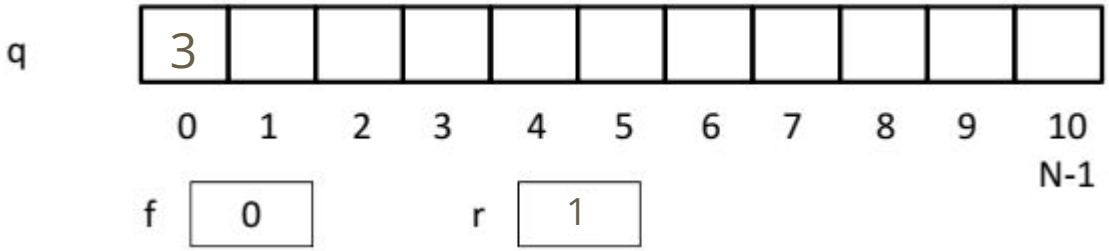
retornar (N-f+r) mod N

Nota: La cola puede almacenar N-1 elementos a lo sumo.

Nota: En este caso, size retorna (N-f+r) % N = (11-0+0)%11 = 11 % 11 = 0

Nota: Otra opción más sencilla es usar un atributo más para representar siz

Cola con arreglo circular: enqueue(e)



enqueue(e)

Si size() == N-1 entonces

error(“cola llena”)

Sino

q[r] = e

r = (r+1) mod N

Nota: La cola puede almacenar N-1 elementos a lo sumo.
r vale 1 porque apunta al siguiente slot vacío

Cola con arreglo circular: dequeue(e)

```
dequeue():
Si isEmpty() entonces
        error( “cola vacía”)
Sino
        temp = q[f];
        q[f] = null;
        f = (f+1) mod N;
        retornar temp
```

Cola con arreglo circular: front(e)

```
dequeue():
Si isEmpty() entonces
        error( “cola vacía”)
Sino
        temp = q[f];
        q[f] = null;
        f = (f+1) mod N;
        retornar temp
```

```
front():
Si isEmpty() entonces
        error( “cola vacía”)
Sino
        retornar q[f]
```

THE END

Bibliografía

Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5